

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Network Throughput and Packet Loss

Pseudocode

ALGORITHM NetworkPerformance

```
BEGIN

    totalDataReceived ← 8000
    time ← 4
    packetsSent ← 200
    packetsReceived ← 180

    // Validation
    IF time = 0 THEN
        DISPLAY "Error: Time cannot be zero."
        STOP
    END IF

    IF packetsSent = 0 THEN
        DISPLAY "Error: Total packets sent cannot be zero."
        STOP
    END IF

    // Calculate throughput
    throughput ← totalDataReceived / time

    // Calculate packet loss
    lostPackets ← packetsSent - packetsReceived
    packetLossPercentage ← (lostPackets / packetsSent) × 100

    // Display results
    DISPLAY "Network Performance Summary"
    DISPLAY "Total Data Received: ", totalDataReceived, " bits"
    DISPLAY "Time: ", time, " seconds"
    DISPLAY "Packets Sent: ", packetsSent
    DISPLAY "Packets Received: ", packetsReceived
    DISPLAY "Packets Lost: ", lostPackets
    DISPLAY "Throughput: ", throughput, " bps"
    DISPLAY "Packet Loss: ", packetLossPercentage, "%"

// Classify network
    IF throughput > 1500 AND packetLossPercentage < 10 THEN
        DISPLAY "Network Classification: Efficient"
    ELSE
        DISPLAY "Network Classification: Inefficient"
    END IF


```

END

Calculation

Throughput:

$$Throughput = \frac{8000}{4} = 2000 \text{ bps}$$

Packets lost:

$$200 - 180 = 20$$

Packet loss percentage:

$$\frac{20}{200} \times 100 = 10\%$$

Since throughput = **2000 bps** > **1500 bps**, but packet loss = **10% is not less than 10%**, the network is:

Classification: Inefficient

2. Bandwidth Utilization and Overloaded Links

Pseudocode:

ALGORITHM MonitorBandwidth

BEGIN

 Read numberOfLinks

 IF numberOfLinks = 0 THEN

 DISPLAY "Error: No network links available."

 STOP

 END IF

 Create array bandwidthUsage[numberOfLinks]

 WHILE TRUE DO

 DISPLAY "Starting bandwidth monitoring..."

 FOR i ← 0 TO numberOfLinks - 1 DO

 // Collect current bandwidth utilization

 bandwidthUsage[i] ← GetBandwidthUtilization(i)

 DISPLAY "Link ", i + 1,

 " Utilization: ",

 bandwidthUsage[i], "%"

 // Check for overload

 IF bandwidthUsage[i] > 80 THEN

 DISPLAY "WARNING: Link ", i + 1,

 " is overloaded."

 DISPLAY "Recommendation: Switch traffic "

 "to backup link."

 SwitchTrafficToBackup(i)

 ELSE

 DISPLAY "Link ", i + 1,

 " is operating normally."

 END IF

 END FOR

 WAIT monitoringInterval

END WHILE

END

How it improves bandwidth management

This algorithm helps the administrator to:

1. **Detect congestion early** before the link becomes severely overloaded.
2. **Distribute traffic** to available backup links.
3. **Reduce packet loss and delay** caused by congestion.
4. **Improve bandwidth utilization** by avoiding unnecessary overloading.
5. **Maintain network performance** through continuous monitoring.

3. Best Path Selection Using Bandwidth, Delay and Utilization

Pseudocode:

ALGORITHM BestPathSelection

BEGIN

Read numberOfLinks

Read numberOfBranches

IF numberOfLinks = 0 OR numberOfBranches = 0 THEN

 DISPLAY "Error: Links and branches cannot be zero."

 STOP

END IF

Create arrays:

 bandwidth[numberOfLinks]

 delay[numberOfLinks]

 utilization[numberOfLinks]

 linkStatus[numberOfLinks]

 qualityScore[numberOfLinks]

WHILE TRUE DO

 DISPLAY "----- Network Monitoring -----"

 // Collect information for every link

 FOR i ← 0 TO numberOfLinks - 1 DO

 bandwidth[i] ← GetBandwidth(i)

 delay[i] ← GetDelay(i)

 utilization[i] ← GetUtilization(i)

 linkStatus[i] ← CheckLinkStatus(i)

 // Check link failure

 IF linkStatus[i] = "FAILED" THEN

 qualityScore[i] ← 0

 DISPLAY "Link ", i + 1, " has FAILED."

 GenerateAlert("Link failure detected")

 CONTINUE

END IF

```

// Check overloaded link
IF utilization[i] > 80 THEN

    DISPLAY "Link ", i + 1, " is OVERLOADED."
    GenerateAlert("Link overloaded")

END IF

// Calculate link quality score
qualityScore[i] ←
    (bandwidth[i] / MaximumBandwidth) × 40
    + ((MaximumDelay - delay[i])
        / MaximumDelay) × 30
    + ((100 - utilization[i]) / 100) × 30

END FOR

// Select best path for each branch
FOR branch ← 1 TO numberOfBranches DO

    bestScore ← -1
    bestPath ← NONE

    FOR each available path from Headquarters
        to branch DO

        pathScore ← 0
        pathAvailable ← TRUE

        FOR each link in path DO

            IF linkStatus[link] = "FAILED" THEN
                pathAvailable ← FALSE
                BREAK
            END IF

            IF utilization[link] > 80 THEN
                pathAvailable ← FALSE
                BREAK
            END IF

            pathScore ← pathScore + qualityScore[link]

        END FOR

        IF pathAvailable = TRUE AND
            pathScore > bestScore THEN

            bestScore ← pathScore
            bestPath ← path

        END IF

    END FOR

END FOR

```

```

// Select or switch path
IF bestPath ≠ NONE THEN

    RouteTraffic(branch, bestPath)

    DISPLAY "Branch ", branch,
        ": Best Path Selected = ", bestPath

    DISPLAY "Path Quality Score = ", bestScore

ELSE

    DISPLAY "No suitable path available for Branch ",
        branch

    GenerateAlert("No reliable path available")

END IF

END FOR

WAIT monitoringInterval

END WHILE

```

END

Link Quality Formula

A weighted score can be used:

$$\begin{aligned}
 \text{QualityScore} &= \text{BandwidthScore}(40\%) + \text{DelayScore}(30\%) \\
 &\quad + \text{UtilizationScore}(30\%)
 \end{aligned}$$

Where:

$$\begin{aligned}
 \text{BandwidthScore} &= \frac{\text{Bandwidth}}{\text{MaximumBandwidth}} \times 40 \\
 \text{DelayScore} &= \frac{\text{MaximumDelay} - \text{Delay}}{\text{MaximumDelay}} \times 30 \\
 \text{UtilizationScore} &= \frac{100 - \text{Utilization}}{100} \times 30
 \end{aligned}$$